TRADING ENGINE · SLICE 2 · DEEP DIVE

Merging live_orders and live_trades into one correct order book

The problem we're about to solve, how the wider industry solves versions of it, the resources worth reading to understand it properly, and the plan for building it — in plain English and diagrams only. No code here: this is the map before you write the territory yourself.

What this document covers

- **The problem** — why two independent, correctly-ordered streams still can't just be processed one after the other
- **The industry answer** — the CS pattern underneath it, and the real systems (market-data vendors, exchanges, stream processors) that solve exactly this
- **Resources** — specific books, specs, and a paper worth actually reading
- **Our plan** — the merge loop's logic, step by step, with the failure modes it has to survive
- **An ordered build plan** — what to write and test first, second, third

Where this picks up
bootstrap() now takes a seeded TradeReconciler

Where this lives
a new function in src/feed/, not apps/

Who writes the code
you — this is design only

SECTION 1

01 The Problem

What we already have

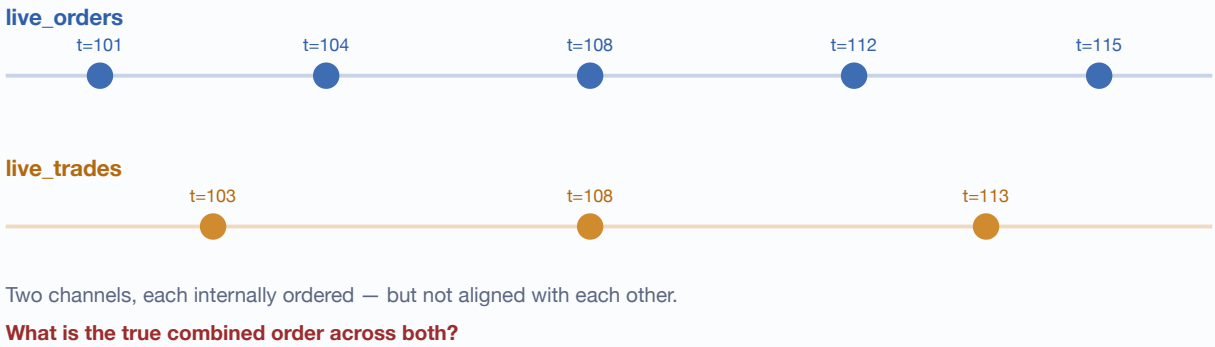
`bitstamp::bootstrap()` takes a REST snapshot and produces a correctly-seeded `OrderBook`, and optionally seeds a `TradeReconciler`'s record at the same time. That's a photograph: it's exactly right for the instant the snapshot was taken, and it is never touched again after that function returns.

A trading session is not a photograph. It's a movie. After the snapshot, Bitstamp keeps sending two independent streams of messages for as long as the session runs: `live_orders` (an order was added, its size changed, or it was removed) and `live_trades` (two specific orders just matched). Keeping the book correct means consuming *both* of these, forever, in the right order.

Two streams, not one

The core wrinkle is that these are genuinely two separate channels. Each one arrives in its own correct internal order — Bitstamp doesn't send you `live_orders` messages out of sequence relative to each other, and the same is true of `live_trades`. But nothing ties the two channels together. Message 40 on `live_orders` and message 12 on `live_trades` could have happened one microsecond apart, in either order, and you only know which by comparing their timestamps — there is no shared sequence number spanning both.

FIGURE 1



Every dot is a real message with its own venue timestamp. Reading top-to-bottom then left-to-right (all orders, then all trades) does not recover the order these actually happened in.

Why not just process one stream fully, then the other?

Because book correctness at any given instant depends on the true combined chronological order, and a trade can matter to a decision your code makes about an order event that, on the wall clock, came right after it. Concretely: if a trade at `t=108` fully consumes a resting order, and an order event at `t=108.5` references a price level that order was sitting in, the level's remaining quantity has to already reflect that consumption before the later event is applied — otherwise the book is wrong for the entire window between those two events, even if it eventually "catches up" once the trade is (very late) processed.

Put differently: correctness isn't just about eventually applying every event. It's about the book being right *at every point along the way*, because a live trading engine has to be able to look at the book at any instant and trust it — not just trust it once a batch finishes.

The two streams don't even use the same operation

This is what makes it more than a plain merge. An order event and a trade event are not interchangeable inputs to one function — they need genuinely different handling:

EVENT TYPE	WHAT HAPPENS TO IT	WHY
Order event	Classify it, then `apply()` to the real book, then (if that succeeded) `observe()` into the reconciler's record.	It's independent new information — the venue is directly telling you the book changed.
Trade event	`reconcile()` against the reconciler's record to get zero or more corrective order events, then `apply()` each one to the real book. Never `observe()` a correction.	It's not new state by itself — it's evidence that might imply state the venue never announced through `live_orders` at all.

WHY THIS ASYMMETRY EXISTS AT ALL

An order that arrives and is matched *immediately* never rests, so it's never announced on `live_orders` — and consequently, the order it consumed sometimes never gets an explicit removal announced either. The only evidence that any of this happened lives on `live_trades`. That's the entire reason `TradeReconciler` exists: to turn trade evidence into the corrective order events `live_orders` should have sent but didn't.

Ties need a defined winner

Two events can legitimately share a timestamp. When that happens, the order event has to win — not arbitrarily, but because `observe()` must update the reconciler's record *before* `reconcile()` reads it for a trade at that same instant. Get this backwards even once, and a correction gets computed against stale state.

FIGURE 2

This is the entire reason for the tie-break rule: it isn't a coin flip, it's forcing `observe()` to run before any `reconcile()` that could depend on it at the same instant.

One stream will end before the other

In a replay against a captured file, one text file will almost always have fewer lines than the other, or its last real event lands before the other's. In a live session, the trade stream could theoretically stop publishing while orders keep flowing (or the reverse). Either way, "compare two heads" stops making sense the moment one side has nothing left to compare — the design has to say explicitly what happens then.

Restating the problem in one sentence

Given two independently-arriving, internally time-ordered streams of trading events — one describing book state, one describing executions — produce a single execution against the book and its reconciler such that, at every instant, the state is exactly what it would have been had every event truly arrived in one global timestamp order, using the correct operation for each event's type, and surviving either stream running out first.

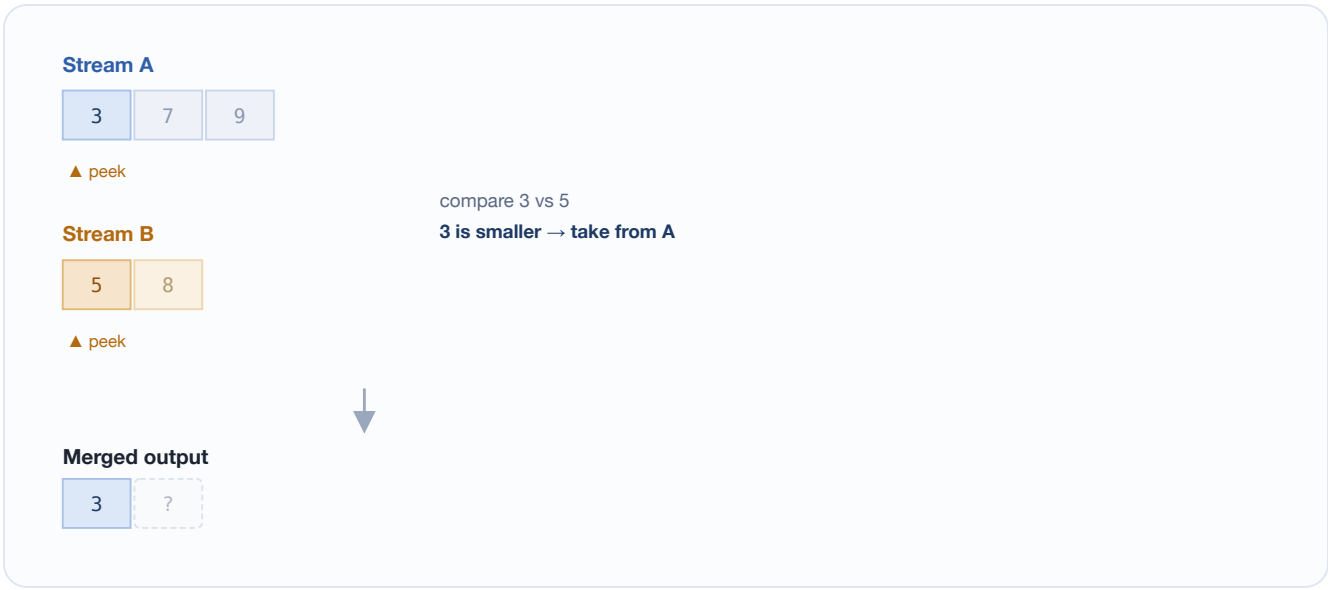
SECTION 2

02 How the Industry Solves This

The CS fundamental underneath it: k-way merge

Strip away the trading domain and this is the classic "merge" step from merge sort, generalized from two sorted lists to *k* sorted streams. The mechanism is always the same three moves: **peek** the next item on every stream, **compare** the peeked items, **advance** whichever stream produced the smallest one. Repeat until every stream is empty.

FIGURE 3



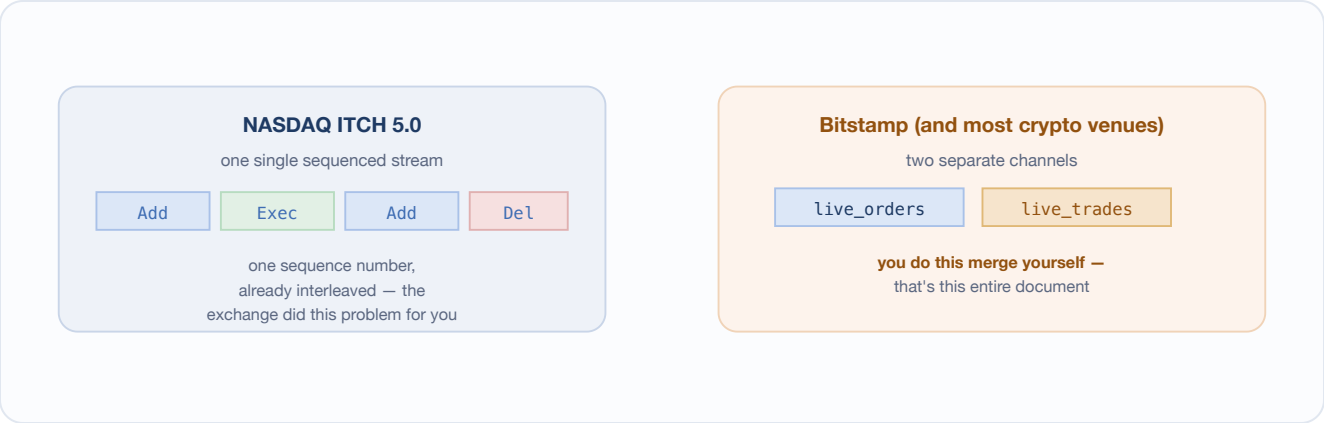
Peek both heads, compare, take the smaller, repeat. This is exactly the merge step inside merge sort — the only thing our version adds is that "take" means something different depending on which stream it came from.

This same two-pointer mechanic, at industrial scale, is how databases perform an external merge sort when data doesn't fit in memory, how a SQL engine's **merge join** combines two pre-sorted tables without hashing either one, and how log-structured storage engines — Cassandra, RocksDB, LevelDB — compact many sorted on-disk files (SSTables) into fewer, larger ones during background compaction. Different domain, identical mechanism.

The market-data version: order book reconstruction from split feeds

Whether this problem exists *at all* depends entirely on how a given venue publishes its data. Some exchanges do the merge for you before you ever see a byte:

FIGURE 4



If this project ran on ITCH, this problem would not exist — the venue already hands you one interleaved sequence. Bitstamp doesn't, so the client is responsible for the join.

This exact gap — "most crypto venues don't pre-merge their public feeds, so someone has to" — is literally the paid product several market-data companies sell: **Tardis.dev**, **Kaiko**, and **Amberdata** all normalize and re-sequence raw exchange feeds into a single clean, correctly-ordered stream, specifically because most venues don't do it themselves. What's being designed in this document is a smaller, single-venue version of the same reconciliation work those companies charge for.

The broker-side analog: reconciling orders against fills

The same shape of problem shows up on the sell side of the industry, not just in market data. A broker's Order Management System receives order acknowledgements and execution reports as two conceptually distinct event types over the FIX protocol — an order's status and a fill against it are not the same message, and they must be joined by order ID to keep one accurate position and book. That's the same "order-state messages vs. execution-evidence messages, joined by ID" shape that `TradeReconciler` already implements in miniature; the merge function extends it to a continuous, live setting.

The stream-processing version: event-time joins and watermarks

Modern stream processors — Apache Flink, Kafka Streams, Google's Dataflow model — formalize this same peek/compare/advance mechanic for arbitrarily many out-of-order streams at scale, under the name **event-time processing**. The extra concept they add is a **watermark**: a declaration of "I don't expect anything older than time T from this stream anymore," which is exactly what "one stream has run out" means in our smaller version — just generalized to streams that might resume, be delayed, or arrive wildly out of order across a distributed system instead of a single ordered file.

Resources worth actually consuming

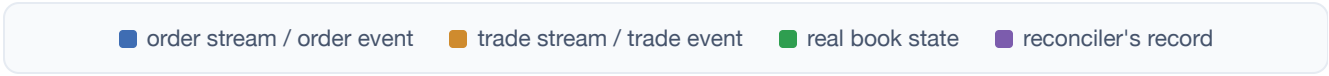
RESOURCE	WHY IT'S WORTH YOUR TIME
Larry Harris — *Trading and Exchanges: Market Microstructure for Practitioners*	The standard text on how real matching engines relate orders and trades, priority rules, and what a trade actually implies about book state. This is the conceptual backbone of everything `TradeReconciler` does.
Maureen O'Hara — *Market Microstructure Theory*	More academic, useful once the mechanics click and you want the theory behind why venues publish data the way they do.
NASDAQ TotalView-ITCH 5.0 specification (public PDF)	Read the message-type section. Seeing a venue that pre-merges order and execution messages into one sequence makes it obvious, by contrast, exactly what work Bitstamp is leaving to you.
LOBSTER (lobsterdata.com) and its methodology paper	An academic tool that reconstructs limit order books from ITCH-style message data. Worth reading for how a mature, published tool thinks about message ordering and book state — even though it starts from an already-merged feed, which is itself an instructive contrast.
Cormen, Leiserson, Rivest, Stein — *Introduction to Algorithms* (CLRS)	The formal treatment of merge sort and k-way merge, if you want the algorithm stripped down to its mathematical bones before reasoning about the trading-specific rules layered on top.
Martin Kleppmann — *Designing Data-Intensive Applications*, ch. 3 and ch. 11	Ch. 3 covers merging sorted structures in storage engines (including LSM-trees); ch. 11 covers stream joins directly. One of the clearest explanations of this pattern generalized well beyond trading.
Akidau, Chernyak, Lax — *Streaming Systems* (O'Reilly)	The canonical book on event-time processing and watermarks. Read this once the two-stream version below feels natural and you want to see how the same idea scales to many streams.
LeetCode #23, "Merge k Sorted Lists"	A pure-mechanics practice problem for exactly the two-pointer merge pattern, with every bit of trading-domain complexity stripped away. Useful as a warm-up if the core mechanic ever feels tangled up with the domain rules.
FIX Trading Community — public FIX protocol specification, `ExecutionReport`	To see the broker-side analog: how order state and fill evidence are kept as distinct message types in one of the industry's most widely used protocols.

SECTION 3

03 How We're Going to Build It

What already exists, and what this function's job is

`bitstamp::bootstrap()` produces a seeded `OrderBook` and, if you pass it a pointer, a seeded `TradeReconciler`. This new function's entire job starts *after* that: given those two already-seeded objects and the rest of the session's `live_orders` / `live_trades` data, keep the book correct. It belongs in `src/feed/`, next to `bootstrap` — not in `apps/` — and it should be provable by its own tests without needing a real socket or the `Feed` interface, the same way `bootstrap()` already is.



What the two inputs should be, for now

There's already a precedent in this codebase for exactly this situation: `test_golden_replay.cpp`'s `replayFromSeedTo` takes a captured event stream as one whole string, and walks it line by line with an `std::istringstream`. The merge function's two inputs — the `live_orders` content and the `live_trades` content — should follow that same convention: plain text in, walked line by line internally. That keeps this fully testable today, against synthetic and captured JSONL, without inventing the `Feed` / `ReplayFeed` / `LiveFeed` abstraction early — that's real, separate, later work (Slices 3 and 5). When a real socket eventually exists, it will feed this same logic real decoded events instead of text lines; the merge logic itself shouldn't need to change.

The main loop, step by step

1 **Peek the next real event on each stream.** For each side, read forward through lines — skipping anything that decodes as `not_order_event` (protocol noise like `bts:subscription_succeeded`) or fails to decode at all — until you have one real candidate event, or the stream has nothing left. This is exactly what the golden replay test already does for a single stream; here it happens independently on both.

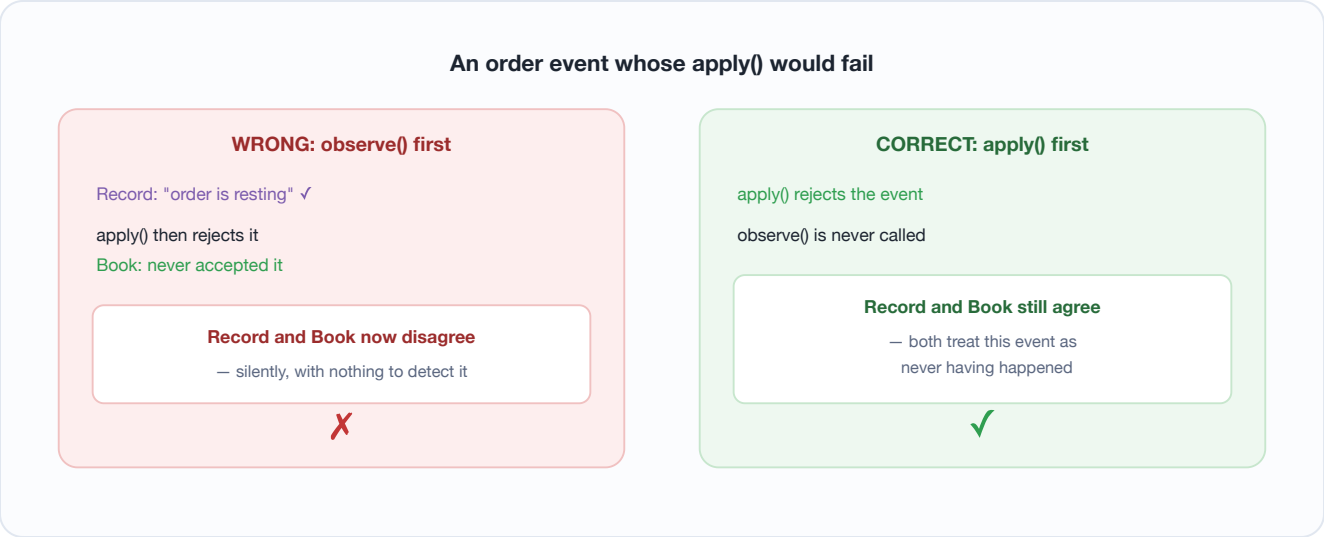
2 **Compare the two candidates.** If both streams produced a candidate, compare their timestamps. The earlier one is consumed this round. On an exact tie, the order event wins — see Figure 2 for why this isn't arbitrary.

3 **Order event wins → classify, then apply, then observe.** Run it through the classifier first (the same zero-price-lifecycle filtering the golden replay test already applies). Then `apply()` it to the real book. **Only if that succeeds**, call `observe()` to update the reconciler's record.

A CORRECTION TO THE EARLIER DESIGN PASS

The original five-rule sketch (from the prior session) said "`observe()`, then `apply()`" for this path. That's revised here to **apply() first, then observe() only on success**. The reasoning was already established when fixing `bootstrap()`'s own seeding loop: `observe()` has no undo. If it ran first and `apply()` then rejected the event, the record would believe an order is resting that the real book never actually accepted — a silent divergence between the two. That was harmless inside `bootstrap()` only because a failure there aborts the whole function. This merge loop keeps running after a single event's failure, so the ordering stops being cosmetic here and becomes load-bearing. See Figure 5.

FIGURE 5



The record and the real book must never be allowed to believe different things. Gating `observe()` on `apply()`'s success is what guarantees that.

4 **Trade event wins → reconcile, then apply each correction.** Run it through `reconcile()`, which reads the record and may return zero, one, or two corrective order events (checking both the trade's buy-side and sell-side order id, since either could be the one that was actually resting). `apply()` each correction returned. **Never** call `observe()` on a correction — `reconcile()` already updated its own record as part of producing it, since a correction describes the record's own conclusion, not new independent information.

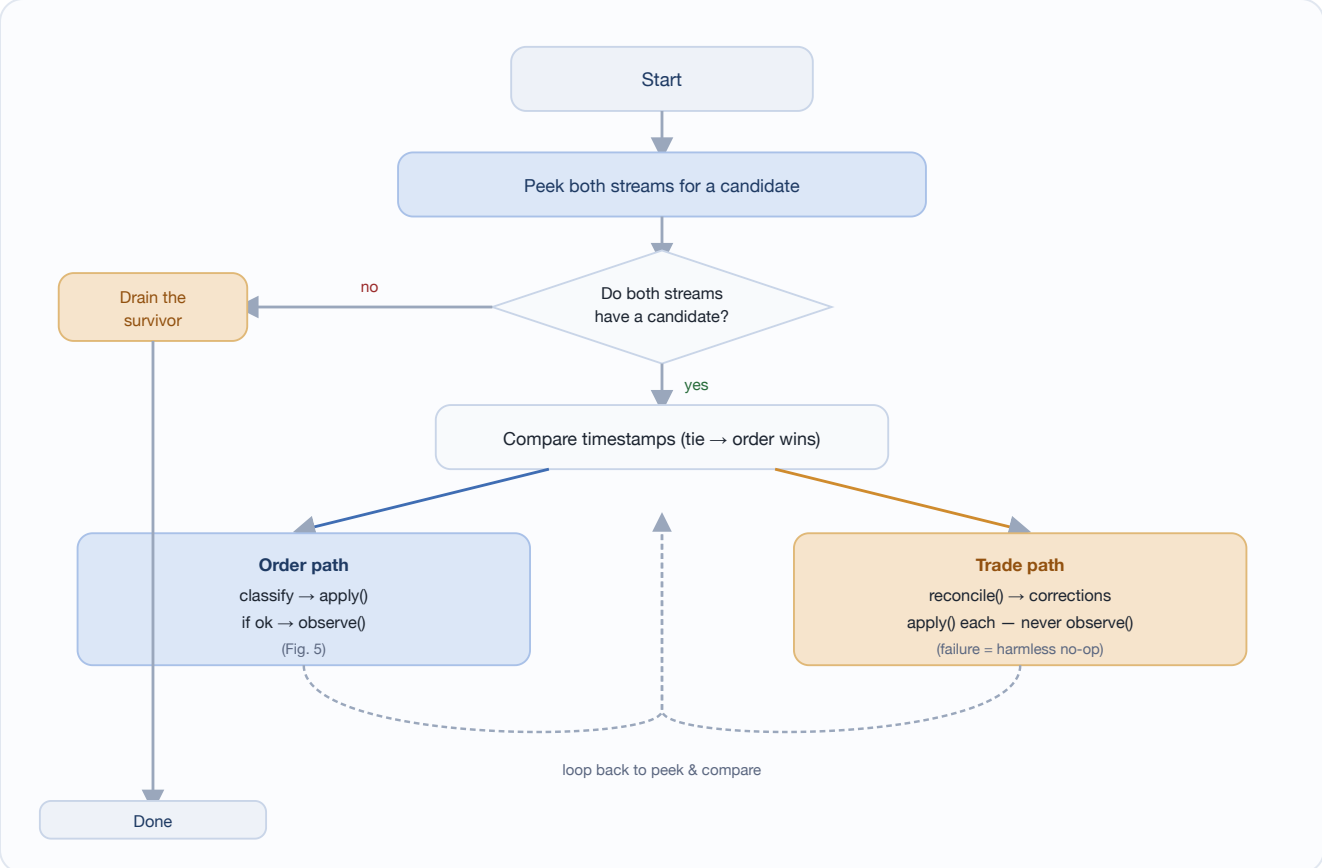
A CORRECTION'S APPLY() FAILING IS EXPECTED, NOT A BUG

If `live_orders` already independently reported the same removal by the time the correction gets applied, `apply()` will fail with `unknown_order_id`. That's fine: it means the book and the record already agree the order is gone — the correction turned out to be redundant, not wrong. Treat it as a harmless no-op at the call site, exactly as `TradeReconciler`'s own class documentation already states.

5 **Loop back to step 1** and peek again, for as many rounds as both streams keep producing candidates.

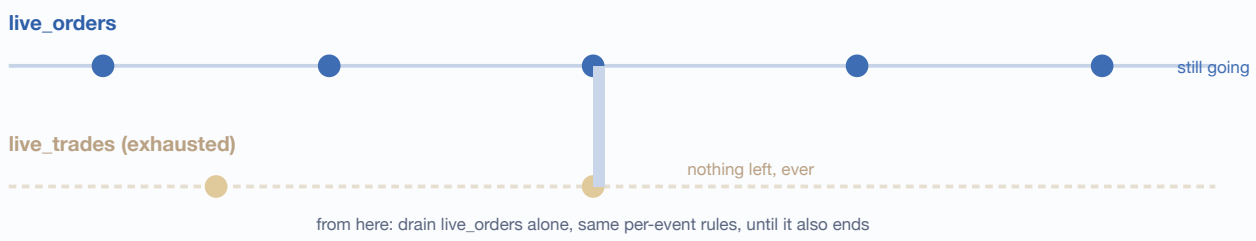
6 **Drain whichever stream survives.** The moment one stream has nothing left (ever — not just "nothing right now"), stop comparing altogether and consume the other stream alone, using the exact same per-event rules, until it's also exhausted.

FIGURE 6 — THE WHOLE LOOP



The whole function is this loop plus a drain tail. Replay against a captured file terminates when both streams are exhausted; a truly live session simply never reaches "no candidate at all" and keeps peeking forever — same loop, different definition of "nothing left *right now*."

FIGURE 7



Draining isn't a special case with different logic — it's the same order-path handling from Figure 6, just running without a trade candidate to compare against each round.

Edge cases the design has to survive

? **Both streams start empty.** The function should do nothing and return cleanly — not a special case, just step 1 immediately finding nothing on either side.

? **A stream is non-empty but has zero real events** — every line is protocol noise. Peeking has to walk all the way through it and correctly conclude "nothing here," not stop at the first non-event line.

? **A trade references an order id neither stream has mentioned yet.** `reconcile()` already handles this — an unknown id simply produces no correction for that side of the trade.

? **The exact-timestamp tie** (Figure 2) — deliberately construct a test for this; it's easy to get right by accident and never notice it was ever ambiguous.

? **A correction's `apply()` fails** with `unknown_order_id` — expected, harmless, must not abort the loop.

? **One stream ends mid-session while the other keeps going** in a genuinely live setting (not a replay file ending). This is explicitly still open — noted below, out of scope for the first version.

EXPLICITLY OUT OF SCOPE FOR THIS FIRST VERSION

Mid-stream degradation — what happens if the trade stream drops out partway through a live session, after already being wired in — is a distinct problem from "no trades stream configured at all" (which passing `nullptr` already handles for free), and it's still unresolved. Don't try to solve it inside this function's first version; note it and move on. Also out of scope: the real `Feed` interface, the SPSC queue, and any actual socket handling — all Slice 3 / Slice 5 work that this function is deliberately built to not need yet.

SECTION 4

04 The Build Order

A suggested order to actually write this in — smallest provable piece first, same discipline the rest of this codebase already follows.

1. **Decide the exact shape of the two inputs first, and write it down.** Two strings? Two `std::istream&`? Decide deliberately and note why — this is the one real design choice left open in Section 3, and it's yours to make.
2. **Write the "peek next real event" logic for one stream, in isolation, before touching the merge loop.** Test it directly: does it correctly skip protocol noise, correctly report "nothing left" at end of input, correctly return the next real event otherwise? Get this right on its own first — it's the piece everything else depends on, and it's much easier to get right without the second stream and the comparison logic complicating it.
3. **Add the timestamp comparison and the two-branch dispatch**, using the peek logic from step 2 on both streams. At this point you're just routing to the right branch — don't implement the branches' bodies yet, stub them.
4. **Implement the order-event branch**: classify, `apply()`, and only on success `observe()`. Test it against an orders-only input with no trades at all — it should behave identically to feeding `OrderBook` directly, since there's nothing for the trade side to do.
5. **Implement the trade-event branch**: `reconcile()`, then `apply()` each correction, tolerating an `unknown_order_id` failure as a no-op. Test it against a trades-only input where nothing is resting — every trade should produce zero corrections.
6. **Implement the drain tail** for whichever stream survives. Test both directions: orders outlasting trades, and trades outlasting orders.
7. **Write the test that actually justifies this whole function existing**: seed a book from a snapshot where one order is resting only in the snapshot, then feed a trade that fully consumes it with no corresponding `live_orders` removal ever appearing. The book should correctly show that order gone. If this test doesn't exist, nothing has actually proven the reconciler helps in the full pipeline — only that it works in isolation, which `test_trade_reconciler.cpp` already covers.
8. **Deliberately construct the exact-tie test** from Figure 2, and a test where a correction's `apply()` is expected to fail harmlessly.
9. **Once the synthetic cases are solid, run it against the real captured segment** — same style as `test_golden_replay.cpp` — now merging both real files instead of just replaying `live_orders` alone.